Chair of Electronic Design Automation
Department of Electrical and Computer Engineering
Technical University of Munich

Lab Manual

Embedded System Design for Machine Learning

Prof. Dr.-Ing. Ulf Schlichtmann
Hon.-Prof. Dr.-Ing. Wolfgang Ecker

Winter semester 2024/25

Teaching Assistants: Philipp van Kempen, M.Sc. &
Samira Ahmadifarsani, M.Sc.

Contents

List of Figures

Lab 0.

Introduction and Preparation

Lab 1.

Model Design, Training and Quantization

1.1. Introduction

In this lab session, you will build your own keyword spotting (KWS) system. At its core, this system contains a convolutional neural network (CNN). In order to design the neural network, you will be using Keras [1]. Keras is a machine learning library written in Python that runs on top of TensorFlow [2]. Keras provides a clean and easy way to create deep learning models. It also leverages various optimization techniques to make high-level neural network design more straightforward and effective.

Your neural network needs to fulfil a number of constraints, such as accuracy and model size. You will need to find a good network architecture that meets all requirements and constraints. After designing and testing your neural network, you will quantize and prepare it for deployment on microcontrollers (which you will do in lab 2). Additionally, this lab session encompasses several theoretical tasks in which you will have to estimate key parameters of neural network layers *by hand*.

The main goal of this laboratory assignment is to familiarize yourself with the general process of designing machine learning, particularly neural network, based models for embedded systems. After completing this lab, you should have a general understanding of data preprocessing pipelines, neural networks, and critical design aspects one needs to consider when deploying to resource-constrained systems.

1.2. Background

This section will introduce the lab-related theoretical concepts that will help you solve the lab tasks. However, because of the breadth of the subject, we will only be able to cover the presented topics from a general point of view. If you feel that you need more in-depth knowledge in certain areas, we encourage you to have a look at the papers and books we cite throughout this section.

1.2.1. Keyword Spotting Pipeline

Figure 1.1 depicts the general pipeline of a modern keyword spotting system using a neural network model as a classifier. It consists of three main parts: an audio feature extractor that converts the incoming audio signal $\mathbf{s}$ into a more compact representation $\mathbf{X}$,

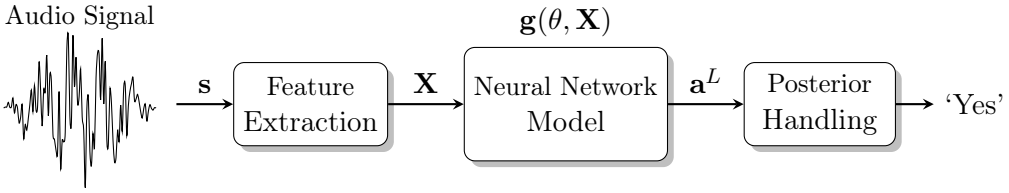


Figure 1.1.: General pipeline of a keyword spotting system.

the neural network-based machine learning model that outputs the category posteriors $\mathbf{g}(\theta, \mathbf{X}) = \mathbf{a}^L$, and a finally a posterior handling process that determines the possible existence of keywords in the audio signal by averaging the outputs of the neural network.

1.2.2. Audio Dataset

For training your neural network, you will be using the *Speech Commands* [3] dataset from Google. This is an open-source collection of more than 100k utterances of 1 s long WAV files [4] sampled at 16 kHz using Pulse-code modulation (PCM) [5]. The dataset has a size of about 3.8 GB and entails 35 different words, like *Yes* or *No*, from over 2k speakers. Additionally, it contains random sounds, like washing machine and car noises, or simply silence. These sounds are important for training as they help the classifier become more robust against background noise. If you are interested in learning more about collecting, cleaning, and processing the audio files, we suggest you look at the great paper from Pete Warden [3].

Keep in mind that while the audio data $\mathbf{s}$ for training in lab 1 consists of short audio sample files, the audio data for inference on the microcontroller in lab 2, later on, will be a continuous stream of data coming in from the microphone.

1.2.3. Feature Extraction

Using $\mathbf{s}$ directly as an input to our neural networks is far from desirably due to a number of reasons: 1) the input vector is too large for our small networks to operate on, 2) the input vector is most likely quite sparse, and 3) the input vector's elements are highly correlated [6]. Thus, we will need to bring the vector into a more *compact* (lower-dimensional), *discriminative* (in terms of audio features), and *robust* (invariant to noise and pitch) form. This is where feature extraction comes into play. Feature extraction, also known as preprocessing or *the frontend*, usually consists of a number of human-engineered signal processing steps that transform the incoming data into a form better suited for our machine learning model. In our case the feature extraction converts the time domain signal $\mathbf{s}$ into a $T \times F$ *mel-scale spectrogram* feature matrix $\mathbf{X}$, with time windows $t = 1, \ldots, T$ and frequency bins $f = 1, \ldots, F$.

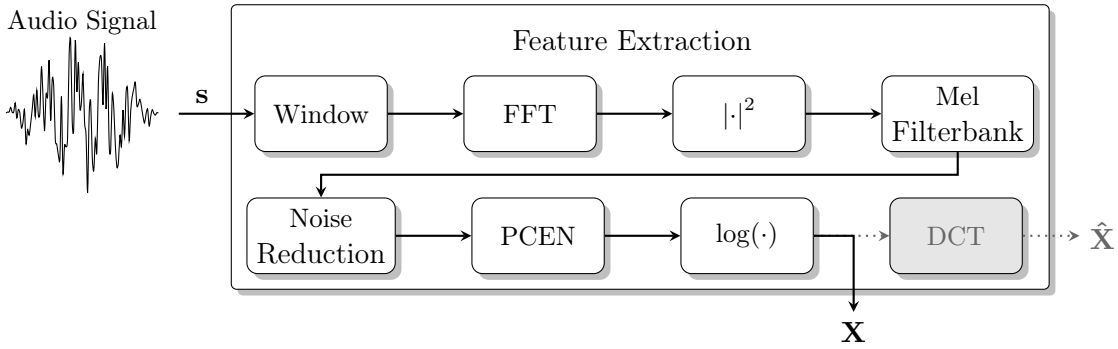


Figure 1.2.: Feature extraction frontend for keyword spotting based on `FrontendProcessSamples()`.

In the following, we will take a closer look at how exactly we extract this feature matrix $\mathbf{X}$ from the audio signal $\mathbf{s}$. Figure 1.2 provides a high-level overview of the different processing steps. You are not required to know every detail by heart, but you should have a general understanding of the feature extraction procedure.

The preprocessing and feature extraction pipeline presented here is based on the `FrontendProcessSamples()` function that resides in the `frontend.c` file. This function is responsible for converting the incoming data $\mathbf{s}$ into a feature matrix $\mathbf{X}$ that the CNN can work with. It gets invoked during training in order to convert the 1 s sample files, as well as during inference on the microcontroller (see lab 2). We will now take a closer look at the individual steps taken during a call to this function.

Windowing

The first step of our feature extraction process is called *windowing*. In this step we cut the 1 s long feature vector $\mathbf{s}$ into smaller overlapping 30 ms audio chunks, with an overlap of 10 ms (see Figure 1.3). This results in $T = 49$ smaller audio vectors $\mathbf{s}_{\{i\}}$

$$T = \lceil \frac{sample_{ms} - window_{ms}}{stride_{ms}} \rceil = \lceil \frac{1000ms - 30ms}{20ms} \rceil = 49.$$

Each of these vectors contains 30 ms × 16 kHz = 480samples. In order to efficiently apply a Fourier transform to them in the next step, we need to zero pad them to the next power of two, i.e. 512. Thus, we concatenate the 480 samples with 32 zero values. Additionally, we apply a so-called Hann window to the sample vector in order to reduce spectral leakage (see Figure A.4).

FFT

After splitting the incoming signal $\mathbf{s}$ into 30ms chunks $\mathbf{s}_{\{i\}}$ with 512 samples each, we apply a Fourier transform to each chunk (see Appendix A.1.2). The Fourier Transform converts signals from the time domain into the frequency domain, exposing the signal's

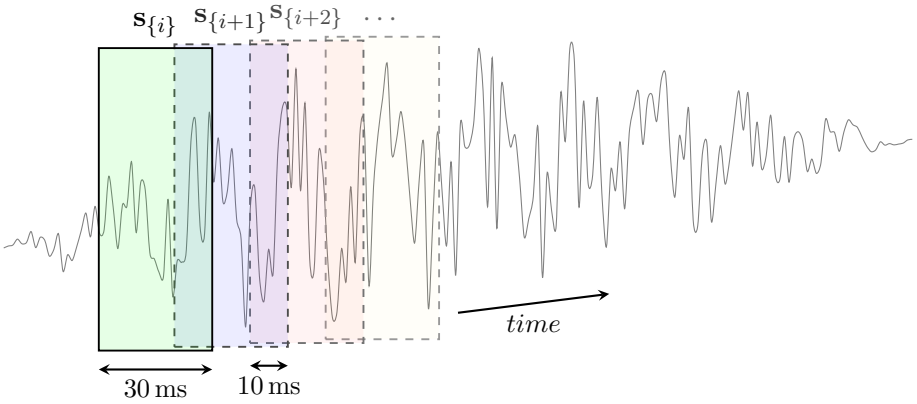


Figure 1.3.: Windowing of audio signal with overlapping windows.

frequency components [7, 8]. Because the signal sample vectors $\mathbf{s}_{\{i\}}$ consist of discrete samples, we need to employ a *discrete Fourier transform* (DFT) (see Appendix A.1.3). Almost all implementations of the DFT rely on the *fast Fourier transform* (FFT) algorithm (see Appendix A.1.4), which is a lower complexity implementation of the DFT, making it practically feasible. The FFT reduces the $\mathcal{O}(N^2)$ complexity of the DFT to $\mathcal{O}(N \log N)$ [7, 8]. This segmentation and windowing procedure combined with applying a Fourier Transform is also known as a *short-time Fourier transform* (STFT) (see Appendix A.1.5)), because it computes a short windowed Fourier Transform [7, 8]. Appendix A.1 provides a more formal overview of the Fourier transform, the DFT, FFT, STFT, and their mathematical relations. You are not required to know every detail, but you should have a general understanding of the time-frequency analysis employed in the frontend.

Absolute Value

After applying the FFT, we get a frequency representation of the original signal chunks. Because we are only working with real values in our application, we discard the upper half of the sample vector and take the absolute values of the remaining 256 samples. This results in a frequency domain vector $\mathbf{x}_{\{i\}}$ of the time domain signal vector $\mathbf{s}_{\{i\}}$. Every entry of $\mathbf{x}_{\{i\}}$ contains the signal's energy at its respective frequency bin. $\mathbf{x}_{\{i\}}$ is also known as the power spectral density (PSD). Putting all $T = 49$ frequency domain vectors $\mathbf{x}_{\{i\}}$ next to each other results in the *raw* frequency domain feature matrix $\mathbf{X}$.

Mel Filterbank

We run each of the the frequency domain signal vectors $\mathbf{x}_{\{i\}}$ through a so-called *Mel filterbank* consisting of 40 filters spaced across the frequency spectrum according to the *Mel-scale* [9]. The filter coefficients are motivated by the nature of the speech signal and the human perception of such signals. The Mel-scale aims to mimic the non-linear human ear perception of sound. It is more discriminative at lower frequencies and less

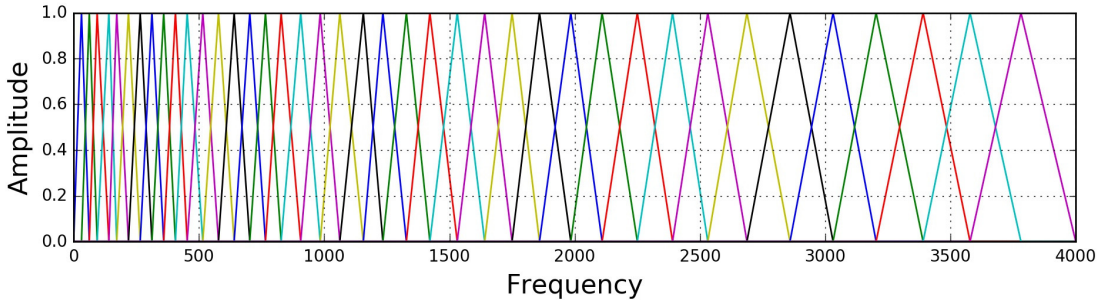


Figure 1.4.: Visual representation of the 40 Mel Frequency bins (frequency range not accurate)[10].

discriminative at higher frequencies. Each filter in the filter bank is triangular, having a response of 1 at the centre frequency and decreasing linearly towards 0 till it reaches the centre frequencies of the two adjacent filters where the response is 0, as shown in Figure 1.4. The filter bank helps to further reduce the number of samples from 256 down to 40. For a compact introduction with information on Mel filterbanks, please have a look at the excellent blog post on *Speech Processing for Machine Learning* by Haytham Fayek [10].

Noise Reduction & PCEN

Keyword spotting systems are designed to work in devices such as smart speakers or in-vehicle systems located at a certain distance from the speaker. This means that communication might occur in far-field conditions, and background noise and reverberation can be particularly harmful due to distance attenuation. Thus, we apply a simple noise reduction, as well as an automatic gain control (AGC) method called PCEN (Per-Channel Energy Normalization) [11]. PCEN helps to manage the signal amplitudes dynamically, depending on the amount of background noise present. The interested reader is referred to [6, 11], as we will not go into any further details here. *Attention:* PCEN is called PCAN in the TensorFlow microfronted code for no apparent reason.

Log Scaling

As a final step, we apply logarithmic scaling to the attained values. This brings the signal amplitudes into a logarithmic scale. This is again motivated by the human perception of audio signal amplitudes and aids in reducing the dynamic range of the features. Furthermore, the multiplicative relation between the excitation signal and impulse response is converted into an addition.

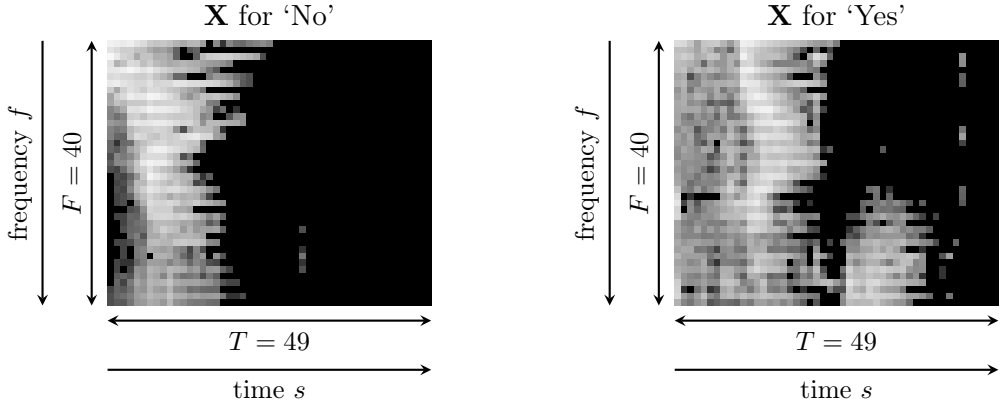


Figure 1.5.: Exemplary feature matrices $\mathbf{X}$ for 'No' and 'Yes'.

DCT

Because the frequency bins in $\mathbf{X}$ are still partially correlated, and in order to further reduce the size of the feature input matrix to the neural network, we could apply a discrete cosine transform (DCT) to decorrelate the filter bank coefficients and yield a compressed representation of the filter banks. After applying the DCT, we would only keep the lowest 2 to 13 bins. Higher frequencies can be discarded as they represent fast changes in the filter bank coefficients, which don't contain any relevant information. These features $\hat{\mathbf{X}}$ are called Mel-frequency cepstral coefficients (MFCCs). MFCCs were very popular with earlier machine learning methods, such as Gaussian Mixture Models / Hidden Markov Models. With the advent of Deep Learning in speech systems, MFCCs are not strictly necessary anymore, as neural networks are less susceptible to correlated inputs. Experimental results show that log-Mel spectra yield approximately the same KWS accuracy as employing full-precision MFCCs [6]. Thus, we abstain from using MFCCs and will not in our further consider them here.

Result

The resulting feature matrix $\mathbf{X}$ thus only contains a total of $T \times F = 49 \times 40 = 1920$ values. This makes it much easier to work with in resource constrained systems. Exemplary feature matrices can be seen in Figure 1.5. What we have essentially done is converted a time domain signal into a frequency *image* on which we can now run our CNN model.

1.2.4. Training and Quantization

TensorFlow Framework and Keras API

Tensorflow is an open-source end-to-end framework for Machine learning capable of running on several architectures. It also provides support for several programming languages. In this lab, we will only interact with the Python interface, which offers integration with interactive Jupyter Notebook sessions.

Keras is a library built on top of TensorFlow 2, which allows to design and train model architectures in a more straightforward manner. Therefore the rest of this paragraph will focus on the important details and some tips on how to get started using the Keras API.

The code to get started with developing a Keras model architecture will be made available together with this lab manual. The individual steps are explained there as well in a more detailed manner.

Example Usage More examples can be found on: `https://keras.io/guides/functional_api/`

- Define model input with Keras: `inputs = keras.Input(shape=(784,))`
- Add a Dense layer: `outputs = layers.Dense(64, activation="relu")(input)`
- Create Keras model:
  `model = keras.Model(inputs=inputs, outputs=outputs, name="simple_model")`
- Display model: `model.summary()`
- Compile model:
  `model.compile(loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True), optimizer=keras.optimizers.RMSprop(), metrics=["accuracy"],)`
- Train/fit model: `history = model.fit(x_train, y_train, batch_size=64, epochs=2, validation_split=0.2)`
- Test model: `test_scores = model.evaluate(x_test, y_test, verbose=2)`
- Save model to disk: `model.save("path_to_my_model")`
- Load model from disk: `model = keras.models.load_model("path_to_my_model")`

Model Quantization with TFLite

The resulting network and the final set of weights are stored in a `SavedModel` format, which is not suitable for execution on an embedded device.

The `.tflite` file format provided by the TensorFlow Lite project is one solution to this problem as it was designed to fulfil the following requirements:

- Light-weight
- Low Latency
- Secure
- Optimal power consumption
- Pre-trained (Inference-only)

Typical targets supported by the TFLite runtime environment are mid-to-high-end microprocessors (e.g. SoCs you can find in a modern smartphone) with multiple heterogeneous/homogenous processor cores and optionally a hardware accelerator for common machine learning tasks built into the SoC module.

TensorFlow Lite for Microcontrollers

TFLite offers an API to convert trained Keras/TensorFlow models into the `.tflite` file format and several options to optimize models on the way. The most important feature is called Post-Training Quantization (POT). It can reduce a model's memory requirements and latency on devices that do not have powerful floating point units. This is achieved by replacing all floating point tensor data by preferably signed `int8`. To convert between the quantized and actual representation of a tensor, additional metadata (e.g. `scale` and `zero_point`) have to be stored alongside the QNN (quantized neural network)[1]:

$$\text{real\_value} = (\text{int8\_value} - \text{zero\_point}) \times \text{scale}$$

The constraints for deploying very small quantized models to a microcontroller platform are even tighter. Therefore a sub-project called TFLite for Microcontrollers (TFLM) [12] was proposed several years ago, shipping with an even more lightweight runtime and kernels which are optimized for systems that do not support dynamic memory allocations very well. TFLM follows an interpreter based approach, which parses the serialized TFLite model from a binary array stored in the MCUs read-only memory. This introduces several overheads:

- Increased initialization time: The time it takes to parse the model graph, including tensors and operators, to build up the model dynamically on the device.
- In addition, memory planning might also take place *on-the-fly* which requires some runtime depending on the used algorithm. Fortunately, the impact of this overhead is neglectable if the device is only booted once to run multiple inferences.
- The code size increases, and the implementation of the interpreter (as well as the parser) have to be added to the software stack running on the target device.
- As hand-crafted generic kernel implementations are linked, the user has to explicitly specify which operator implementations should be linked with the compiled binary to keep the program size low.

The overcome these overheads, a different approach will be used for the model deployment in the second lab assignment.

[1] See `https://www.tensorflow.org/lite/performance/post_training_quantization`

Model Compression

While quantization can be seen as a Model compression technique, some approaches can reduce the complexity of a model even more to achieve a better inference latency or minimal memory requirements. Two prominent examples are:

- Exploiting Tensor Sparsity via pruning: Increasing the number of zeros in the model's weights to store the model data more efficiently.
- Using *sub-byte* quantization techniques: i.e. 5 bit for constant weights and activations.

To effectively make use of these techniques during inference, some sort of encoding has to be defined and supported by the kernels and or runtime environment. This encoding may introduce additional overheads, which make it not profitable to apply such compression methods to all operators in a model. Ideally, the target processor or accelerator has some hardware to handle the compressed tensors, which may result in a decreased inference latency.

Another aspect which has to be considered is the compression typically has to be considered already during the training procedure as just truncating the constants in the pre-trained model would result in a degradation of model accuracy.

Estimation of Model Complexity

An important skill for designing *TinyML*-compatible models is having an awareness of the constraints given by the application, target device, and environment. Especially memory requirements and the available processing power are limiting factors for the model architecture design space. In this lab, you will learn how to estimate the ROM and RAM usage, as well as the number of MAC operations in a simple convolutional neural network (CNN). This way, you will not waste time training and testing models which are not suitable for running your target device. You should be able to evaluate general design decisions using ballpark numbers and intuition in order to reduce design time.

Memory requirements for deploying a model to an embedded device can be split up into two categories:

1. ROM footprint: Mostly constant tensor weights, but also the kernel (operator implementations) definitions. Additional runtime-related overheads, as well as application specific code (Peripheral Drivers, RTOS,...), have to be considered.
2. RAM usage: This involves mostly intermediate tensor buffers (activations) but also temporary scratch pad memory required by certain kernels. Memory planning which can happen during runtime (online) or statically (offline), has a big impact on the RAM footprint of a model. Again additional application- or framework-specific overheads might be non-negligible.

There exists a number of different metrics to evaluate the performance of deployed models, such as statistic time measurements to determine the number of inferences per second. As this topic is very complex (processor microarchitecture, memory architecture etc., have to be considered), we will instead primarily rely on relative measurements. In a typical CNN, the most time is spent for *Multiply and Accumulate (MAC)* operations. As you will learn during the lab 1 assignment, those can be counted reasonably quickly. Fortunately, convolutional and dense layers are very linear in a way that a change in the size of an input or kernel/filter is reflected in the number of MACS in a proportional manner.

1.3. Assignments

1.3.1. Setup

1. Clone/Download `micro-kws` repository.
2. Setup the tools (Python Environment, Jupyter (Notebook/Lab) Server) as explained in `1_train/README.md` and in the "Lab Setup" manual.
3. Enter the names of students and assigned list of keywords in the respective files `student/names.txt` and `student/words.txt`

1.3.2. Tasks

1. Training and Quantization of a MicroKWS model build with Keras
   a) Follow the steps described in `1_train/MicroKWS.ipynb` using the provided settings
   b) Complete the following programming-related exercises:
      i. Update `student/callbacks.py` to define a EarlyStopping[2] callback with Keras which stops the training procedure after 10 or more epochs of neglectible improvement of the `val_loss` quantity.
      ii. Update `student/metrics.py` to calculate the per-class metrics `recall`, `precision` and `f1_score` for a given confusion matrix.
      iii. Derive formulas to estimate the complexity of a given TFLite model in terms of ROM/RAM usage as well as number of MACs (Multiply-Accumulate Operations) and update `student/estimate.py` accordingly.
   c) Update the used keywords for the model to the ones assigned to your group (See Moodle) and the used model architecture to `micro_kws_student`. (See `FLAGS.model_name` and `FLAGS.wanted_words`)
   d) **Optional:** Design your own model architecture for the keyword-spotting task which satisfies each of the following constraints to get 5 bonus points.
      i. Accuracy of the quantized TFLite model is at least 90%.
      ii. Total memory requirement to store all constants/weights in ROM is at most 75 kB.
      iii. Best case memory requirement for intermediate tensors in RAM is at most 50 kB.
      iv. Estimated number of MAC operations is at most 10^7 (10M).
      v. You are not allowed to change anything (learning parameters etc.) except the Keras code in `student/model.py` used to define the model. In addition, variations of the provided example model are not accepted, thus we expect you to add at least one more layer to the model.

[2] `https://keras.io/api/callbacks/early_stopping/`

e) At the end of this section your should have the following files
   - `student/names.txt` & `student/words.txt`
   - `student/callbacks.py` & `student/metrics.py`
   - `student/estimate.py` & `student/model.py`
   - `models/micro_kws_student_{your_keywords}.tflite`
   - `models/micro_kws_student_{your_keywords}_quantized.tflite`

   which will be used to grade the submission.

f) Create submission ZIP archive using the provided `submit.py` script.

g) Upload your submission ZIP archive to Moodle.

1.3.3. Deliverables

Moodle Upload:

- Single ZIP archive: `submission_1.zip` (generated using `1_train/submit.py`)
- Multi-file submissions are not accepted!
- To access the upload form, you have to pick a group first.
- Only one group member has to press the submit button. The other group members should be able to see/update the submission.

1.3.4. Hints

- For programming exercises:
  - Only modify the code sections which are highlighted as follows:
    `### ENTER STUDENT CODE BELOW ###`
  - Inline comments and docstrings are provided in the respective Python files to help you with your implementation.
  - To check the functionality of your code before the final submission, some basic unit tests are provided in the `1_train/tests/` directory. (See `1_train/README.md` for details)
- For the estimation tasks:
  - Memory alignment requirements can be ignored.
  - For an good estimation of the quantized model, you will have to consider the bias and weights separately
  - Assumptions: Neither branches nor nodes with multiple inputs/outputs exist in the trained model. The graph is processed in a linear way so that at most 2 buffers will be used at the same time.

- For the final challenge:
  - To keep the complexity low, you should stick the common layer types such as Dense, (depthwise) 2D convolutions, Pooling, Reshape/Flatten and activation functions.

1.3.5. Testing implementation before final submission

We provide some basic unit-tests which should allow you to test your solution. These tests are a subset of the (larger) test-suite which is used for grading the lab. To run the tests, you need to execute `python -m pytest tests/` (or `!python -m pytest tests/` in the Jupyter notebook) in the `micro-kws/1_train` directory. More details can be found in the README file. Figures 1.6a and 1.6b show how the execution of test should look like.

```
=========================== short test summary info ============================
FAILED tests/test_callbacks.py::test_callbacks_early_stopping - assert None
FAILED tests/test_estimate.py::test_estimate_macs - assert 0 == 1
FAILED tests/test_estimate.py::test_estimate_rom - AssertionError: assert 0 == 160
FAILED tests/test_estimate.py::test_estimate_ram - AssertionError: assert 0 == 36864
FAILED tests/test_metrics.py::test_metrics_precision - TypeError: unsupported operan
d type(s) for -: 'float' and 'NoneType'
FAILED tests/test_metrics.py::test_metrics_recall - TypeError: unsupported operand t
ype(s) for -: 'float' and 'NoneType'
FAILED tests/test_metrics.py::test_metrics_f1_score - TypeError: unsupported operand
 type(s) for -: 'float' and 'NoneType'
============================== 7 failed in 9.48s ===============================
```

(a) All tests failed (or unimplemented).

```
collected 7 items

tests/test_callbacks.py .                                                [ 14%]
tests/test_estimate.py ...                                               [ 57%]
tests/test_metrics.py ...                                                [100%]

============================== 7 passed in 2.67s ===============================
```

(b) All tests passed successfully.

Figure 1.6.: Running Unit Tests for Lab 1.

1.3.6. Common Problems

First check the FAQ section in the Lab Setup manual, which contains solutions for the most common setup-related questions, such as connection issues.

- Notebook freezes during execution.
  Solution: Restart Jupyter Kernel. If this does not help, try running the Notebook without a GPU by running the following code before starting the Jupyter Server:

```
export CUDA_VISIBLE_DEVICES=-1
```

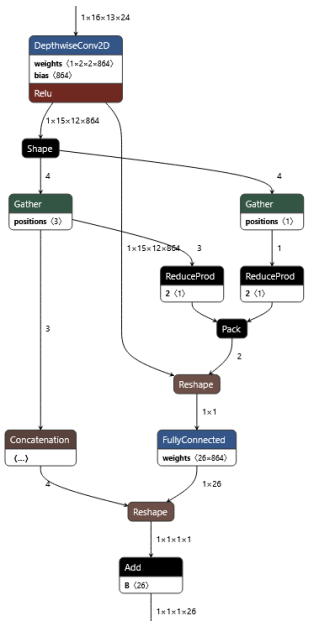
- Randomly crashing unit tests

  Possible Solutions:

- Make sure to use a virtual Python environment where only ESD4ML lab Python requirements are installed. Using the system- or user-wide python installation often leads to random incompatibilities.
- Try running the problematic unit test file on the command line (eventually without GPU) e.g. using:

```
CUDA_VISIBLE_DEVICES=-1 python -m pytest tests/test_metrics.py
```

- TFLite Model Graph looks messy (see Netron screenshot below)



Possible Solution: Make sure to add a Flatten/Reshape layer in between Conv2D and Dense/FullyConnected layers to convert the 4-dimensional outputs into the 2-inputs expected by the following layer.

Lab 2.

Model Deployment to an Embedded Platform

Lab 3.

Vectorized Multiply

Appendix A.

Mathematical Background

A.1. Spectral Estimation

The goal of *spectral estimation* (or *spectral density estimation*) is to estimate the spectral density of a signal from a sequence of samples. The spectral density characterizes the frequency content of the signal (over time) [13]. The following section is mostly based on [7, 8, 14, 15]. While some parts of this short summary make use of *complex* numbers, the signals encountered in our lab are exclusively real numbered.

A.1.1. Fourier Series

The *Fourier series* in engineering contexts is a method to decompose a periodic signal into its frequency components. It is intimately related to the geometry of infinite-dimensional function spaces, also known as *Hilbert spaces*. Hilbert spaces generalize the notion of vector spaces to functions with infinitely many dimensions [7, 8]. The inner product in Euclidean vector spaces for two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^N$ is defined as

$$\mathbf{a} \cdot \mathbf{b} = \langle \mathbf{a}, \mathbf{b} \rangle = \sum_{i=1}^{N} a_i b_i = a_1 b_1 + a_2 b_2 + \ldots + a_N b_N. \tag{A.1}$$

The notion of inner products can be extended to functions. With complex functions $x(t)$ and $y(t)$ defined on domain $t \in L$ [7, 14]

$$\langle x(t), y(t) \rangle = \int_L x(t) y^*(t) dt. \tag{A.2}$$

Just as in Euclidean vector spaces, the inner product in function spaces may be used to project a function into a new coordinate system. Instead of projecting a vector into orthogonal basis vectors, the function is projected into an orthogonal basis of functions spanning the function space [8]. In the case of infinite-dimensional function spaces, the basis one is projecting into is infinite. This observation directly results in the Fourier series for complex T-periodic functions on $[0, T)$

$$\begin{aligned} x(t) &= \sum_{k=-\infty}^{\infty} c_k \cdot e^{i \frac{2\pi k t}{T}} \\ &= \sum_{k=-\infty}^{\infty} (\alpha_k + i\beta_k)(\cos(\frac{2\pi k t}{T}) + i \sin(\frac{2\pi k t}{T})), \end{aligned} \tag{A.3}$$

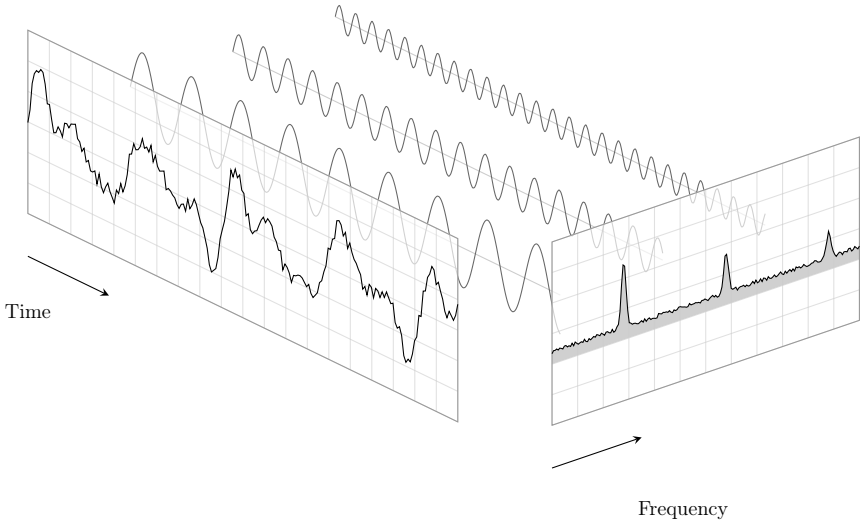


Figure A.1.: Visual representation of Fourier analysis [16].

where $c_k = \alpha_k + i\beta_k$ and $e^{i\gamma} = \cos(\gamma) + i\sin(\gamma)$. Here, the function x is the sum of orthogonal sine and cosine functions. The complex coefficients c_k can be found by applying the above mentioned inner product with the basis function $y_k(t) = e^{i\frac{2\pi kt}{T}}$

$$c_k = \frac{1}{T}\langle x(t), e^{i\frac{2\pi kt}{T}}\rangle = \frac{1}{T}\int_T x(t)\cdot e^{-i\frac{2\pi kt}{T}}dt. \tag{A.4}$$

A.1.2. Fourier Transform

The Fourier series is defined only for periodic functions. To also study non-period functions, the *Fourier transform* is introduced. It is an extension of the Fourier series that results when the period of the function is allowed to approach infinity [7, 14]. The Fourier transform integral is essentially the limit of a Fourier series as the domain's length goes to infinity [7, 8]. Letting $T \to \infty$ and defining $\omega_0 = \frac{2\pi}{T}$ yields

$$\begin{aligned} x(t) &= \lim_{T\to\infty}\sum_{k=-\infty}^{\infty} c_k e^{i\frac{2\pi kt}{T}} \\ &= \lim_{T\to\infty}\sum_{k=-\infty}^{\infty}\left(\frac{1}{T}\int_T x(t)\cdot e^{-i\frac{2\pi kt}{T}}dt\right)e^{i\frac{2\pi kt}{T}} \\ &= \lim_{w_0\to 0}\sum_{k=-\infty}^{\infty}\frac{\omega_0}{2\pi}\left(\int_{-\frac{\pi}{\omega_0}}^{\frac{\pi}{\omega_0}} x(t)\cdot e^{-ik\omega_0 t}dt\right)e^{ik\omega_0 x} \\ &= \lim_{w_0\to 0}\frac{1}{2\pi}\sum_{k=-\infty}^{\infty}\hat{x}\cdot e^{ik\omega_0 t}\omega_0. \end{aligned} \tag{A.5}$$

The discrete product $k\omega_0$ becomes a continuous frequency variable $k\omega_0 \to \omega$, and the summation becomes a Riemann integral. This results in

$$x(t) = \mathcal{F}^{-1}\{\hat{x}(\omega)\}(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} \hat{x}(\omega)e^{i\omega t}d\omega \tag{A.6}$$

$$\hat{x}(\omega) = \mathcal{F}\{x(t)\}(\omega) = \int_{-\infty}^{\infty} x(t)e^{-i\omega t}dt, \tag{A.7}$$

where $\mathcal{F}(\cdot)$ and $\mathcal{F}^{-1}(\cdot)$ are the Fourier transform and inverse Fourier transform, respectively. Both functions converge as long as $\int_{-\infty}^{\infty}|x(t)|dt < \infty$ and $\int_{-\infty}^{\infty}|\hat{x}(\omega)|d\omega < \infty$ [7, 8]. In engineering terms, this is equivalent to saying that the energy of the function (or the signal) needs to be finite.

Parsevals theorem states that the energy of signals is preserved up to a constant when applying the Fourier transform [7, 14]

$$\int_{-\infty}^{\infty}|x(t)|^2dt = \frac{1}{2\pi}\int_{-\infty}^{\infty}|\hat{x}(\omega)|^2d\omega, \tag{A.8}$$

which is an important property when dealing with signal features in the frequency domain.

The *power spectral density* (PSD) $S(f)$ of $x(t)$, with $\omega = 2\pi f$, describes the power distribution over the frequency and can be estimated by taking the square of the absolute value of the Fourier transform

$$\mathrm{S}_x(f) = |\hat{x}(f)|^2. \tag{A.9}$$

The PSD can also be expressed in terms of the Fourier transform of the autocorrelation function

$$\mathrm{R}_{xx}(\tau) = \mathrm{E}[x(t)^* x(t-\tau)] \tag{A.10}$$

$$\mathrm{S}_x(f) = \int_{-\infty}^{\infty}\mathrm{R}_{xx}(\tau)e^{-i2\pi f\tau}dt = \hat{\mathrm{R}}_{\mathrm{xx}}(f), \tag{A.11}$$

where x is assumed to be *wide-sense stationary* (WSS); this relation is known as the *Wiener–Khinchin theorem* [14].

A.1.3. Discrete Fourier Transform

The *discrete Fourier transform* (DFT) is a discretized version of the Fourier transform, which operates on vectors of data $\mathbf{x} = [x_1, x_2, \ldots, x_N]^{\mathrm{T}}$ that are obtained by sampling continuous functions (or signals) [8].

Closely related and of paramount importance to signal discretization is the so-called *Nyquist–Shannon* sampling theorem. It states that any signal sampled at frequency f_s must contain no sinusoidal component at exactly or above half the sample frequency f_s. Strictly speaking, any signal with bandwidth B has to be sampled at $f_s > 2B$, where the threshold $2B$ is called *Nyquist rate* [14].

One can transform N discrete samples of function x from the time domain to an N-point discrete spectral representation with the DFT, defined as

$$\hat{x}_k = \sum_{j=1}^{N} x_j e^{-i\frac{2\pi jk}{N}}. \tag{A.12}$$

where k is a variable of discrete frequency. Its inverse, the iDFT, is given by

$$x_n = \frac{1}{N} \sum_{j=1}^{N} \hat{x}_j e^{i\frac{2\pi jn}{N}}. \tag{A.13}$$

Comparing these transforms to the above presented inner products of vectors shows that the DFT is a linear transformation that maps the data points of x (time-domain) to $\hat{x}$ (frequency-domain). Thus, this transformation can be computed by matrix multiplication [8]

$$\begin{bmatrix} \hat{x}_1 \\ \hat{x}_2 \\ \hat{x}_3 \\ \vdots \\ \hat{x}_N \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_N & \omega_N^2 & \cdots & \omega_N^{N-1} \\ 1 & \omega_N^2 & \omega_N^4 & \cdots & \omega_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{N-1} & \omega_N^{2(N-1)} & \cdots & \omega_N^{(N-1)^2} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_N \end{bmatrix} \tag{A.14}$$

with the DFT matrix of size $N \times N$, where $\omega_N = e^{\frac{-2\pi i}{N}}$.

The discrete version of Parseval's theorem, which states that the total signal energy is preserved by the DFT, is

$$\sum_{n=1}^{N} |x_n|^2 = \frac{1}{N} \sum_{k=1}^{N} |\hat{x}_k|^2. \tag{A.15}$$

The discrete PSD is defined as

$$S_{x,k} = \frac{1}{N} |\hat{x}_k|^2, \tag{A.16}$$

where, depending on the context, $1/N$ is sometimes left out.

A.1.4. Fast Fourier Transform

Multiplying by the DFT matrix involves $\mathcal{O}(N^2)$ operations, making it too slow to be practically applicable. This is where the *fast Fourier transform* (FFT) comes in [17]. Popularized by J. W. Cooley and John Tukey, it scales with $\mathcal{O}(N \log N)$, enabling it to be used in many real-time applications [18]. The fundamental idea of the FFT is to subdivide the large DFT matrix recursively into smaller matrices. This can be done most

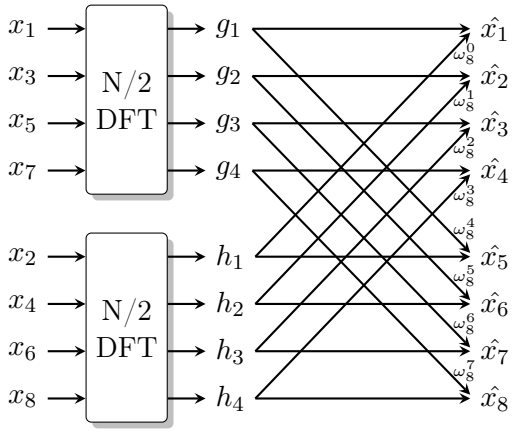


Figure A.2.: 8-point Radix-2 FFT high-level signal flow.

efficiently if N is a power of 2, which yields the so-called *radix-2 algorithm*. The even and odd entries of the N-long input get split into two smaller $N/2$-long DFTs

$$\begin{aligned}
\hat{x}_k &= \sum_{j=1}^{N} x_j e^{-i\frac{2\pi jk}{N}} \\
&= \sum_{j=1}^{N/2} x_{2j} e^{-i\frac{2\pi 2jk}{N}} + \sum_{j=1}^{N/2} x_{2j+1} e^{-i\frac{2\pi(2j+1)k}{N}} \\
&= \underbrace{\sum_{j=1}^{N/2} x_{2j} e^{-i\frac{2\pi jk}{N/2}}}_{N/2 \text{ DFT}} + \underbrace{e^{-i\frac{2\pi k}{N}}}_{\omega_N^n} \underbrace{\sum_{j=1}^{N/2} x_{2j+1} e^{-i\frac{2\pi jk}{N/2}}}_{N/2 \text{ DFT}}
\end{aligned} \tag{A.17}$$

which for $N = 8$ can be visualized at the highest recursion level as seen in Figure A.2.

Repeating this process recursively for any input size down to $N = 2$, computing all DFTs on the way down, and then bringing the results back together yields the improved run-time FFT. It is important to remember that the FFT and the DFT return the same result; the FFT simply improves the computational complexity for large N.

The FFT is implemented in various software packages. The most relevant implementations are the FFTPACK implementation, written in FORTRAN [19], which is used by NumPy [20] and SciPy [21]. There also exists the FFTW implementation, which is written in C [22]. The FFTW uses advanced heuristics to achieve superb performance, making it faster than the FFTPACK implementation. However, TensorFlow Lite Micro makes use of a much simpler implementation, called KISS ("Keep It Simple, Stupid") FFT [23].

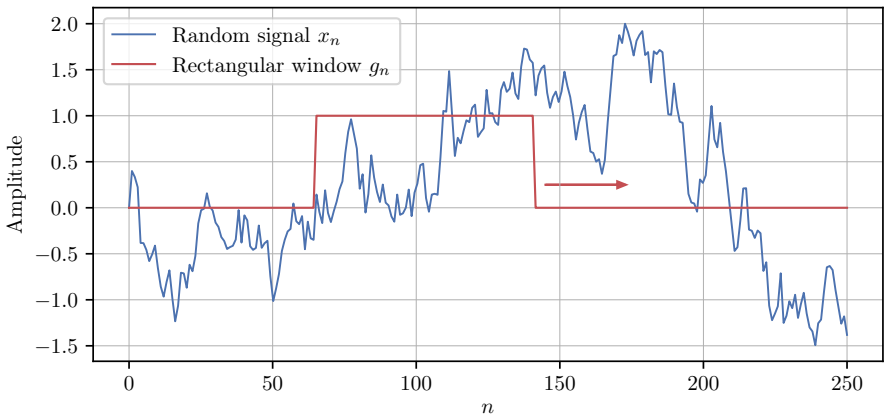


Figure A.3.: Illustration of the STFT with rectangular sliding window.

A.1.5. Short-Time Fourier Transform

While the Fourier transform provides detailed information on the frequency content of a given signal, it does not yield any information about when in time those frequencies occur. Thus, is it only able to characterize truly periodic and stationary signals. However, for signals with non-stationary frequency content, it is important to simultaneously characterize the frequency and its evolution over time [8].

The *short-time Fourier transform* (STFT) computes windowed DFTs using a moving window $g(t)$. This results in a series of single DFTs, each having the length of the window function. The STFT provides localization in both the temporal domain, as well as in the frequency domain. It assumes that the signal is stationary during the application of the window function to each chunk of the signal, a property called *quasi-stationary*. While this assumption does generally not hold, it results in sufficiently good spectral estimation results. The continuous STFT pair can be written as

$$\hat{x}(\omega, t) = \int_{-\infty}^{\infty} x(\tau) e^{-i\omega\tau} g^*(\tau - t) d\tau \tag{A.18}$$

$$x(t) = \frac{1}{2\pi \|g\|^2} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \hat{x}(\omega, \tau) e^{i\omega t} g(\tau - t) d\omega d\tau, \tag{A.19}$$

where $\|g\|^2$ is the energy of the window function [7, 14]. While the summation, as stated above, goes from $-\infty$ to ∞, the finite window reduces the summation range to the window size. It is important to note that the STFT $\hat{x}(\omega, t)$ now has a frequency ω *and* a time t variable.

The discrete counterpart of the STFT is defined as

$$\hat{x}_{m,k} = \sum_{n=m-L/2}^{m+L/2} x_{n+mL}\, e^{-i\frac{2\pi k n}{L}} g_n \tag{A.20}$$

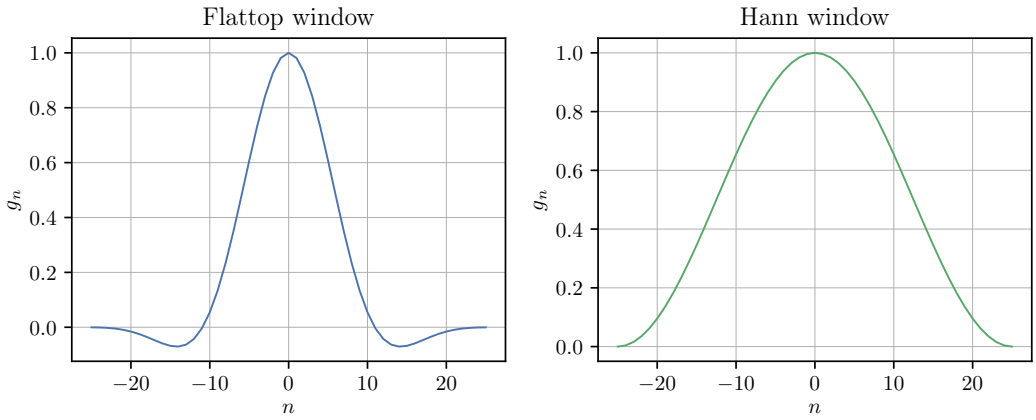


Figure A.4.: Flattop and Hann window with size $n = 50$.

with discrete window function g_n of even size L, so that $k \in [-L/2, L/2]$ becomes the discrete-frequency variable and $m \in [0, \lfloor N/L - 1 \rfloor]$ the discrete-time variable with which one selects the DFT-bin of interest [7]. Thus, for an N entry discrete sample vector x, one has $\lfloor N/L \rfloor$ DFT-bins.

The simplest window function is the *rectangular window*. It is equivalent to replacing all but L values of the data samples in x by zeros, making it appear as though the signal suddenly turns *on* and *off* [13]. It is an example of a window with high-frequency resolution but low dynamic range. Of particular interest to us is the *Hann window*. It presents a compromise between dynamic range and resolution. The Hann window is the window function used by the feature extractor in our lab. It is defined as

$$w[n] = 0.5\left(1 - \cos\left(\frac{2\pi\hat{n}}{N}\right)\right) \qquad 0 \leq n < N, \tag{A.21}$$

where $\hat{n} = n + 0.5$ in order for each point to centered.

The board topic of window functions would go beyond the scope of this lab manual. For more information about window functions, in conjunction with the important topic of spectral leakage, consult [13, 24, 25].

An important tuning factor is the size of the window function; and likewise the length of DFT. One cannot simultaneously localize a signal x in both the time domain and frequency domain. The shorter the DFT, the finer the time resolution. At the same time, the frequency resolution decreases. When increasing the DFT length, the time resolution decreases, and the frequency resolution increases. This observation leads to the fundamental uncertainty principle of time-frequency analysis that limits the ability to simultaneously attain high resolution on both the time and frequency domains. Stated mathematically: for the continuous function $x(t)$, the time-frequency uncertainty

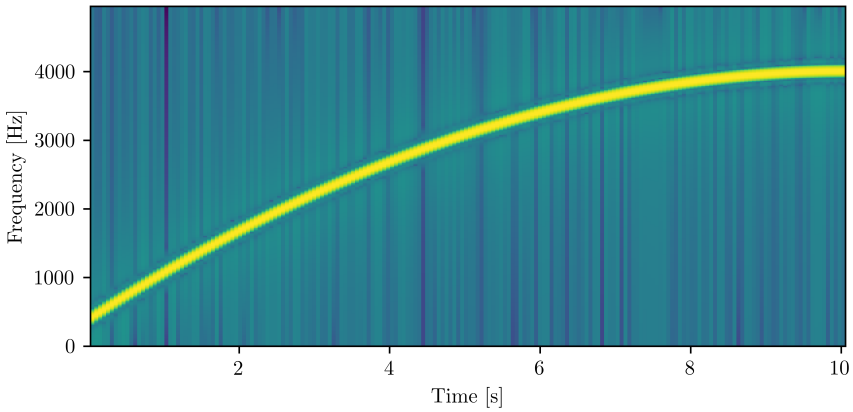


Figure A.5.: Spectrogram over 10s of a quadratic chirp signal from 400Hz to 4000Hz.

principle can be written as

$$\left(\int_{-\infty}^{\infty} t^2|x(t)|^2dt\right)\left(\int_{-\infty}^{\infty} \omega^2|\hat{x}(\omega)|^2d\omega\right) \geq \frac{1}{16\pi^2}. \tag{A.22}$$

if both $tx(t)$ and $x'(t)$ are square integrable [8].

The *spectrogram* (or sometimes called *waterfall plot*) is a two-dimensional representation of the squared magnitude of the STFT. It is essentially a series of PSD absolute values indexed by time. The discrete spectrogram is defined as

$$\mathrm{S}_{m,k} = |\hat{x}_{m,k}|^2 \tag{A.23}$$

with m as the discrete-time variable and k as the index of discrete frequency. A typical format is a two-dimensional graph; one axis represents time, and the other axis represents frequency. The amplitude of a specific frequency at a given time is expressed by the strength or color of each point in the graph (see Figure A.5, for example) [26].

Bibliography

[1] Francois Chollet et al. *Keras*. https://keras.io. 2015.

[2] The TensorFlow Authors. *TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems*. Software available from tensorflow.org. 2015. URL: https://www.tensorflow.org/.

[3] Pete Warden. "Speech commands: A dataset for limited-vocabulary speech recognition". In: *arXiv preprint arXiv:1804.03209* (2018).

[4] Wikipedia. *WAV — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/WAV. [Online; accessed 20-April-2022]. 2022.

[5] Wikipedia. *Pulse-code modulation — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Pulse-code_modulation. [Online; accessed 20-April-2022]. 2022.

[6] Ivan Lopez-Espejo et al. "Deep Spoken Keyword Spotting: An Overview". In: *IEEE Access* (2021).

[7] R. Hoffmann and M. Wolff. *Intelligente Signalverarbeitung 1: Signalanalyse*. Online access with purchase: Springer. Springer Berlin Heidelberg, 2014. ISBN: 9783662453230. URL: https://books.google.de/books?id=QHC1BQAAQBAJ.

[8] Steven L. Brunton and J. Nathan Kutz. *Data-Driven Science and Engineering: Machine Learning, Dynamical Systems, and Control*. Cambridge University Press, 2019. DOI: 10.1017/9781108380690.

[9] Wikipedia. *Mel scale — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/Mel_scale. [Online; accessed 20-April-2022]. 2022.

[10] Haytham Fayek. *Speech Processing for Machine Learning*. 2016. URL: https://haythamfayek.com/2016/04/21/speech-processing-for-machine-learning.html.

[11] Yuxuan Wang et al. "Trainable frontend for robust and far-field keyword spotting". In: *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE. 2017, pp. 5670–5674.

[12] Robert David et al. "TensorFlow lite micro: Embedded machine learning for tinyml systems". In: *Proceedings of Machine Learning and Systems* 3 (2021), pp. 800–811.

[13] Wikipedia. *Window function — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Window%20function&oldid=1000798643. [Online; accessed 24-January-2021]. 2021.

[14] Clemens Gühmann. *Messdatenverarbeitung*. Fachgebiet Elektronische Mess- und Diagnosetechnik, Technische Universität Berlin, 2011.

[15] Petre Stoica, Randolph L Moses, et al. "Spectral analysis of signals". In: (2005).

[16] Tobit Flatscher. *Schematic Fourier Analysis*. 2019. URL: https://tex.stackexchange.com/a/513431 (visited on 12/20/2020).

[17] James W Cooley and John W Tukey. "An algorithm for the machine calculation of complex Fourier series". In: *Mathematics of computation* 19.90 (1965), pp. 297–301.

[18] Marc Alexa. *Wissenschaftliches Rechnen*. Fachgebiet Computer Graphics, Technische Universität Berlin, 2020.

[19] Paul N Swarztrauber. "Vectorizing the ffts". In: *Parallel computations*. Elsevier, 1982, pp. 51–83.

[20] NumPy Contributors. "Array programming with NumPy". In: *Nature* 585 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[21] SciPy Contributors. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[22] Matteo Frigo and Steven G Johnson. "The design and implementation of FFTW3". In: *Proceedings of the IEEE* 93.2 (2005), pp. 216–231.

[23] Mark Borgerding. *KISS FFT*. https://github.com/mborgerding/kissfft. 2022.

[24] Gerhard Heinzel, Albrecht Rüdiger, and Roland Schilling. "Spectrum and spectral density estimation by the Discrete Fourier transform (DFT), including a comprehensive list of window functions and some new at-top windows". In: (2002).

[25] Siemens. *Window Types: Hanning, Flattop, Uniform, Tukey, and Exponential*. 2020. URL: https://community.sw.siemens.com/s/article/window-types-hanning-flattop-uniform-tukey-and-exponential (visited on 11/15/2020).

[26] Wikipedia. *Spectrogram — Wikipedia, The Free Encyclopedia*. http://en.wikipedia.org/w/index.php?title=Spectrogram&oldid=1001441730. [Online; accessed 22-February-2021]. 2021.